

INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE COMPUTO

Arreglos y Apuntadores

Practica 6

Alumno:

Guzmán Celaya Fernando

Grupo: 1CV1

Unidad de aprendizaje:

Fundamentos De Programación

Arreglos y apuntadores

Comenzando se realizó un arreglo y se accedió a este de la forma tradicional y posteriormente se realizó utilizando apuntadores para poder ver el espacio de memoria posteriormente se realizó un cambio para acceder al valor con apuntadores, finalmente se realizó un último cambio que mostraba su dirección en la memoria y el elemento almacenado.

```
// Includes
#include <stdio.h>

int main()
{
    // Variables
    int arr[5] = {5, 4, 3, 2, 1};

    // With usual for
    for (int i = 0; i < 5; i++)
    {
        printf("%d\t %d\n", i, arr[i]);
    }

    // With pointer
    for (int i = 0; i < 5; i++)
    {
        printf("%d\t %d\n", i, (&arr)[i]);
    }

    return 0;
}
```

```
// Includes
#include <stdio.h>

int main()
{
    // Variables
    int arr[5] = {5, 4, 3, 2, 1};

    // With usual for
    printf("Sin apunadores\n");
    for (int i = 0; i < 5; i++)
    {
        printf("%d\t %d\n", i, arr[i]);
    }

    // With pointer
    printf("Con apunadores\n");
    for (int i = 0; i < 5; i++)
    {
        printf("%d\t %d\n", i, *(arr + i));
    }

    return 0;
}
```

```
// Includes
#include <stdio.h>

int main()
{
    // Variables
    int arr[5] = {5, 4, 3, 2, 1};

    // With usual for
    printf("Sin apunadores\n");
    for (int i = 0; i < 5; i++)
    {
        printf("%d\t %d\n", i, arr[i]);
    }

    // With pointer
    printf("Con apunadores\n");
    for (int i = 0; i < 5; i++)
    {
        printf("%d\t %d\t [%u]\n", i, *(arr + i), arr + i);
    }

    return 0;
}
```

Las ejecuciones son las correspondientes:

```
escom@L-1105-3 MINGW64 /e/Programming/Codes
● $ gcc aparr.c -o ./compiled/aparr.out

escom@L-1105-3 MINGW64 /e/Programming/Codes
● $ ./compiled/aparr.out
0      5
1      4
2      3
3      2
4      1
0      6422276
1      6422296
2      6422316
3      6422336
4      6422356
```

```

escom@L-1105-3 MINGW64 /e/Programming/Codes
● $ gcc aparr.c -o ./compiled/aparr.out

escom@L-1105-3 MINGW64 /e/Programming/Codes
● $ ./compiled/aparr.out
Sin apuntadores
0      5
1      4
2      3
3      2
4      1
Con apuntadores
0      5
1      4
2      3
3      2
4      1

```

```

escom@L-1105-3 MINGW64 /e/Programming/Codes
● $ gcc aparr.c -o ./compiled/aparr.out

escom@L-1105-3 MINGW64 /e/Programming/Codes
● $ ./compiled/aparr.out
Sin apuntadores
○ 0      5
1      4
2      3
3      2
4      1
Con apuntadores
0      5      [6422276]
1      4      [6422280]
2      3      [6422284]
3      2      [6422288]
4      1      [6422292]

```

Posteriormente como siguiente código se realizó uno donde se mostrará el paso de elementos usando apuntadores, para esto se realizó una suma de la forma tradicional y posteriormente usando apuntadores

```

// Includes
#include <stdio.h>

void SumaTr(int a, int b, int c)
{
    printf("C antes: %d\n", c);
    c = a + b;
    printf("C despues: %d\n", c);
}

void SumaAp(int a, int b, int *c)
{
    printf("C antes: %d\n", *c);
    *c = a + b;
    printf("C despues: %d\n", *c);
}

int main()
{
    // Variables
    int x = 10, y = 15, z = 1606;

    // Output
    printf("Tradicional\n");
    printf("Z antes:%d\n", z);
    SumaTr(x, y, z);
    printf("Z despues:%d\n", z);

    printf("Apuntadores\n");
    printf("Z antes:%d\n", z);
    SumaAp(x, y, &z);
    printf("Z despues:%d\n", z);

    return 0;
}

```

Su ejecución correspondiente es:

```
escom@L-1105-3 MINGW64 /e/Programming/Codes
● $ gcc aparr.c -o ./compiled/aparr.out

escom@L-1105-3 MINGW64 /e/Programming/Codes
● $ ./compiled/aparr.out
Tradicional
○ Z antes:1606
  C antes: 1606
  C despues: 25
  Z despues:1606
Apuntadores
  Z antes:1606
  C antes: 1606
  C despues: 25
  Z despues:25
```

Posteriormente se realizó una función encargada de cambiar el valor del array haciendo uso de apuntadores

```
// Includes
#include <stdio.h>

void Cambia(int *arr, int n)
{
    for (int i = 0; i < n; i++)
    {
        *(arr + i) = 0;
    }
}

int main()
{
    // Variables
    int arr[5] = {5, 4, 3, 2, 1};

    // Output
    Cambia(arr, 5);

    for (int i = 0; i < 5; i++)
    {
        printf("%d\t %d\n", i, arr[i]);
    }

    return 0;
}
```

Su ejecución es la correspondiente

```
escom@L-1105-3 MINGW64 /e/Programming/Codes
● $ gcc aparr.c -o ./compiled/aparr.out

escom@L-1105-3 MINGW64 /e/Programming/Codes
● $ ./compiled/aparr.out
0      0
1      0
2      0
3      0
4      0
```

Se realizó otra función la cual retornaba la cantidad de caracteres en una línea de texto, posteriormente se realizó la misma funcionalidad de esta

función en otra usando recursividad y apuntadores, por ultimo se realizo dos pequeñas alternativas (funciones) que realizaran la misma función:

```
// Includes
#include <stdio.h>
int LongitudCadena(char *cadena)
{
    // Variables
    int aux = 0;
    for (int i = 0; cadena[i] != '\0'; i++)
    {
        aux++;
    }

    return aux;
}

int main()
{
    // Variables
    char *nombre = "Marco Antonio";
    int cantidad = LongitudCadena(nombre);
    printf("%s tiene %d caracteres\n", nombre, cantidad);
}
```

```
int LongitudCadenaR(char *cadena)
{
    if (*cadena == '\0')
    {
        return 0;
    }

    return 1 + LongitudCadenaR(cadena + 1);
}
```

```

// Includes
#include <stdio.h>

int LongitudCadena(char *cadena)
{
    // Variables
    int aux = 0;
    for (int i = 0; cadena[i] != '\0'; i++)
    {
        aux++;
    }

    return aux;
}

int LongitudCadenaR(char *cadena)
{
    if (*cadena == '\0')
    {
        return 0;
    }

    return 1 + LongitudCadenaR(cadena + 1);
}

int main()
{
    // Variables
    char *nombre = "Marco Antonio";
    int cantidad = LongitudCadena(nombre);
    printf("Resultado sin recursividad\n");
    printf("%s tiene %d caracteres\n", nombre, cantidad);

    printf("Resultado con recursividad\n");
    int cantidadR = LongitudCadenaR(nombre);
    printf("%s tiene %d caracteres\n", nombre, cantidadR);
}

```

```

int LongitudCadenaAlt(char *cadena)
{
    int n = 0;
    for (; *cadena != '\0'; cadena++, n++)
        return n;
}

int LongitudCadenaAlt2(char *cadena)
{
    char *p = cadena;
    while (*p != '\0')
    {
        p++;
    }
    return p - cadena;
}

int main()
{
    // Variables
    char *nombre = "Marco Antonio";
    int cantidad = LongitudCadena(nombre);
    printf("Resultado sin recursividad\n");
    printf("%s tiene %d caracteres\n", nombre, cantidad);

    printf("Resultado con recursividad\n");
    int cantidadR = LongitudCadenaR(nombre);
    printf("%s tiene %d caracteres\n", nombre, cantidadR);

    printf("Resultado con alternativa 1\n");
    int cantidadAlt = LongitudCadenaAlt(nombre);
    printf("%s tiene %d caracteres\n", nombre, cantidadAlt);

    printf("Resultado con alternativa 2\n");
    int cantidadAlt2 = LongitudCadenaAlt2(nombre);
    printf("%s tiene %d caracteres\n", nombre, cantidadAlt2);
}

```

Su correspondientes ejecuciones son las siguientes:

```
escom@L-1105-3 MINGW64 /e/Programming/Codes
• $ gcc aparr.c -o ./compiled/aparr.out

escom@L-1105-3 MINGW64 /e/Programming/Codes
• $ ./compiled/aparr.out
Marco Antonio tiene 13 caracteres
```

```
escom@L-1105-3 MINGW64 /e/Programming/Codes
• $ gcc aparr.c -o ./compiled/aparr.out

escom@L-1105-3 MINGW64 /e/Programming/Codes
• $ ./compiled/aparr.out
Resultado sin recursividad
Marco Antonio tiene 13 caracteres
Resultado con recursividad
Marco Antonio tiene 13 caracteres
```

```
escom@L-1105-3 MINGW64 /e/Programming/Codes
• $ gcc aparr.c -o ./compiled/aparr.out

escom@L-1105-3 MINGW64 /e/Programming/Codes
• $ ./compiled/aparr.out
Resultado sin recursividad
Marco Antonio tiene 13 caracteres
Resultado con recursividad
Marco Antonio tiene 13 caracteres
Resultado con alternativa 1
Marco Antonio tiene 0 caracteres
Resultado con alternativa 2
Marco Antonio tiene 13 caracteres
```

Finalmente se realizaron los ejemplos vistos el día 17 de diciembre, los cuales eran funciones encargadas del copiado de una cadena a otra usando apuntadores y distintas formas, desde la más sencilla hasta aquella que utilizaba notación especial.

```
#include <stdio.h>

void copiaCadena(char *src, char *dest) {
    for (int i = 0; src[i] != '\0'; i++) {
        // printf("%c", src[i]);
        dest[i] = src[i];
    }
}

int main() {
    char a[14] = "Marco Antonio", b[14];

    copiaCadena(a, b);
    printf("Cadena original: %s\n", a);
    printf("Cadena copiada: %s\n", b);

    return 0;
}
```

```
#include <stdio.h>

void copiaCadena(char *src, char *dest) {
    for (int i = 0; src[i] != '\0'; i++) {
        // printf("%c", src[i]);
        dest[i] = src[i];
    }
}

void copiaCadenaV2(char *src, char *dest) {
    int i = 0;
    while (src[i] != '\0') {
        dest[i] = src[i];
        i++;
    }
}

int main() {
    char a[14] = "Marco Antonio", b[14];

    copiaCadena(a, b);
    printf("COPIA V1\n");
    printf("Cadena original: %s\n", a);
    printf("Cadena copiada: %s\n", b);

    printf("\nCOPIA V2\n");
    copiaCadenaV2(a, b);
    printf("Cadena original: %s\n", a);
    printf("Cadena copiada: %s\n", b);

    return 0;
}
```

```

void copiaCadenaV3(char *src, char *dest) {
    int i = 0;
    while (*(src + i) != '\0') {
        *(dest + i) = *(src + i);
        i++;
    }
}

```

```

int main() {
    char a[14] = "Marco Antonio", b[14];

    copiaCadena(a, b);
    printf("COPIA V1\n");
    printf("Cadena original: %s\n", a);
    printf("Cadena copiada: %s\n", b);

```

```

    printf("\nCOPIA V2\n");
    copiaCadenaV2(a, b);
    printf("Cadena original: %s\n", a);
    printf("Cadena copiada: %s\n", b);

```

```

    printf("\nCOPIA V3\n");
    copiaCadenaV3(a, b);
    printf("Cadena original: %s\n", a);
    printf("Cadena copiada: %s\n", b);

```

```

    return 0;
}

```

```

void copiaCadenaV4(char *src, char *dest) {
    int i = 0;
    while ((*dest = *src) != '\0') {
        i++;
    }
}

```

```
void copiaCadenaV5(char *src, char *dest) {  
    while ((*dest = *src) != '\0') {  
        src++;  
        dest++;  
    }  
}
```

```
void copiaCadenaV6(char *src, char *dest) {  
    while ((*dest++ = *src++) != '\0')  
        ;  
}
```

Las ejecuciones correspondientes son las siguientes:

```
○ gcc examples.c -o examples.out
```

```
© ./examples.out
```

```
Cadena original: Marco Antonio
```

```
Cadena copiada: Marco Antonio
```

```
© gcc examples.c -o examples.out
```

```
© ./examples.out
```

```
COPIA V1
```

```
Cadena original: Marco Antonio
```

```
Cadena copiada: Marco Antonio
```

```
COPIA V2
```

```
Cadena original: Marco Antonio
```

```
Cadena copiada: Marco Antonio
```

```
© gcc examples.c -o examples.out
```

```
© ./examples.out
```

```
COPIA V1
```

```
Cadena original: Marco Antonio
```

```
Cadena copiada: Marco Antonio
```

```
COPIA V2
```

```
Cadena original: Marco Antonio
```

```
Cadena copiada: Marco Antonio
```

```
COPIA V3
```

```
Cadena original: Marco Antonio
```

```
Cadena copiada: Marco Antonio
```

```
© gcc examples.c -o examples.out
```

```
© ./examples.out
```

```
COPIA V1
```

```
Cadena original: Marco Antonio
```

```
Cadena copiada: Marco Antonio
```

```
COPIA V2
```

```
Cadena original: Marco Antonio
```

```
Cadena copiada: Marco Antonio
```

```
COPIA V3
```

```
Cadena original: Marco Antonio
```

```
Cadena copiada: Marco Antonio
```

```
COPIA V4
```

```
Cadena original: Marco Antonio
```

```
Cadena copiada: Marco Antonio
```

```
© gcc examples.c -o examples.out
```

```
© ./examples.out
```

```
COPIA V1
```

```
Cadena original: Marco Antonio
```

```
Cadena copiada: Marco Antonio
```

```
COPIA V2
```

```
Cadena original: Marco Antonio
```

```
Cadena copiada: Marco Antonio
```

```
COPIA V3
```

```
Cadena original: Marco Antonio
```

```
Cadena copiada: Marco Antonio
```

```
COPIA V4
```

```
Cadena original: Marco Antonio
```

```
Cadena copiada: Marco Antonio
```

```
COPIA V5
```

```
Cadena original: Marco Antonio
```

```
Cadena copiada: Marco Antonio
```

```
© gcc examples.c -o examples.out
```

```
© ./examples.out
```

```
COPIA V1
```

```
Cadena original: Marco Antonio
```

```
Cadena copiada: Marco Antonio
```

```
COPIA V2
```

```
Cadena original: Marco Antonio
```

```
Cadena copiada: Marco Antonio
```

```
COPIA V3
```

```
Cadena original: Marco Antonio
```

```
Cadena copiada: Marco Antonio
```

```
COPIA V4
```

```
Cadena original: Marco Antonio
```

```
Cadena copiada: Marco Antonio
```

```
COPIA V5
```

```
Cadena original: Marco Antonio
```

```
Cadena copiada: Marco Antonio
```

```
COPIA V6
```

```
Cadena original: Marco Antonio
```

```
Cadena copiada: Marco Antonio
```